7. Test Scenarios and Test Cases

Overview

Test Scenarios and Test Cases are important components of the software testing process. A Test Scenario defines a high-level functionality that needs to be tested, while a Test Case provides detailed steps, test data, expected results, and validation criteria for verifying that functionality.

In this project, test scenarios and test cases were designed for all major modules of the Fake Store API, including Authentication, Products, Categories, Cart, and Users. The purpose was to validate that each API endpoint performs the expected operation and returns accurate responses.

The test cases cover various CRUD operations (Create, Read, Update, Delete), response validations, status code validations, and business logic verification.

Test Scenario 1: User Authentication

Objective

To verify that a registered user can successfully log in and receive an authentication token.

Test Case 1.1 – Verify Login with Valid Credentials

API Endpoint:

```
POST /auth/login
```

Test Steps:

1. Open Postman.
2. Select Login User API request.
3. Enter valid username and password.
4. Click Send.

Test Data:

```
{
  "username": "mor_2314",
  "password": "83r5^_"
}
```

Expected Result:

- Status Code should be 200.

- Authentication token should be generated.
- Response body should not be empty.

Test Scenario 2: Retrieve Product Information

Objective

To verify that product information can be retrieved successfully.

Test Case 2.1 – Verify Get All Products

API Endpoint:

```
GET /products
```

Test Steps:

1. Open Get All Products request.
2. Click Send.

Expected Result:

- Status Code should be 200.
- Product list should be displayed.
- Response should contain multiple product records.

Test Case 2.2 – Verify Get Single Product

API Endpoint:

```
GET /products/1
```

Test Steps:

1. Open Get Single Product request.
2. Enter Product ID.
3. Click Send.

Expected Result:

- Status Code should be 200.
- Correct product details should be returned.

- Product ID should match requested ID.

Test Scenario 3: Create Product

Objective

To verify that a new product can be added successfully.

Test Case 3.1 – Verify Add New Product

API Endpoint:

```
POST /products
```

Test Steps:

1. Open Add Product request.
2. Enter product details.
3. Click Send.

Sample Test Data:

```
{
  "title": "Laptop",
  "price": 45000,
  "description": "Gaming Laptop",
  "image": "https://i.pravatar.cc",
  "category": "electronics"
}
```

Expected Result:

- Status Code should be 200 or 201.
- Product should be created successfully.
- Response should contain newly created product details.

Test Scenario 4: Update Product Information

Objective

To verify that existing product details can be updated successfully.

Test Case 4.1 – Verify Product Update Using PUT

API Endpoint:

```
PUT /products/1
```

Test Steps:

1. Open Update Product request.
2. Modify all product fields.
3. Click Send.

Expected Result:

- Status Code should be 200.
- Updated product information should be returned.
- All fields should be updated.

Test Case 4.2 – Verify Product Update Using PATCH

API Endpoint:

```
PATCH /products/1
```

Test Steps:

1. Open PATCH request.
2. Modify selected fields.
3. Click Send.

Expected Result:

- Status Code should be 200.
- Selected fields should be updated.
- Other fields should remain unchanged.

Test Scenario 5: Delete Product

Objective

To verify that an existing product can be deleted successfully.

Test Case 5.1 – Verify Delete Product

API Endpoint:

```
DELETE /products/1
```

Test Steps:

1. Open Delete Product request.
2. Enter Product ID.
3. Click Send.

Expected Result:

- Status Code should be 200.
- Product should be deleted successfully.
- Response should indicate successful deletion.

Test Scenario 6: Category Management

Objective

To verify category-related APIs.

Test Case 6.1 – Verify Get All Categories

API Endpoint:

```
GET /products/categories
```

Expected Result:

- Status Code should be 200.
- Category list should be displayed.

Test Case 6.2 – Verify Get Products by Category

API Endpoint:

```
GET /products/category/electronics
```

Expected Result:

- Status Code should be 200.
- Products belonging to electronics category should be displayed.

Test Scenario 7: Cart Management

Objective

To verify shopping cart functionality.

Test Case 7.1 – Verify Get All Carts

API Endpoint:

```
GET /carts
```

Expected Result:

- Status Code should be 200.
- Cart records should be displayed.

Test Case 7.2 – Verify Get Single Cart

API Endpoint:

```
GET /carts/1
```

Expected Result:

- Correct cart information should be returned.

Test Case 7.3 – Verify Add to Cart

API Endpoint:

```
POST /carts
```

Expected Result:

- Cart should be created successfully.
- Status Code should be 200.

Test Case 7.4 – Verify Update Cart Using PUT

API Endpoint:

```
PUT /carts/1
```

Expected Result:

- Complete cart details should be updated.

Test Case 7.5 – Verify Update Cart Using PATCH

API Endpoint:

```
PATCH /carts/1
```

Expected Result:

- Selected cart fields should be updated.

Test Case 7.6 – Verify Delete Cart

API Endpoint:

```
DELETE /carts/1
```

Expected Result:

- Cart should be deleted successfully.

Test Scenario 8: User Management

Objective

To verify user information APIs.

Test Case 8.1 – Verify Get All Users

API Endpoint:

```
GET /users
```

Expected Result:

- Status Code should be 200.
- User list should be displayed.

Test Case 8.2 – Verify Get Single User

API Endpoint:

```
GET /users/1
```

Expected Result:

- Correct user information should be returned.
- User ID should match requested ID.

Test Case Design Approach

The following validations were included in all test cases:

Functional Validation

- API performs intended operation.

Status Code Validation

- Correct HTTP status code is returned.

Response Body Validation

- Response contains expected data.

Response Time Validation

- API responds within acceptable time.

JSON Structure Validation

- Response follows expected JSON format.

Data Integrity Validation

- Response data matches request data.

Summary

A total of multiple test scenarios and detailed test cases were designed and executed for Authentication, Products, Categories, Cart, and Users modules. These test cases ensured complete validation of CRUD operations, response accuracy, status codes, response structure, and business functionality. The execution of these test cases helped verify the reliability and correctness of the Fake Store REST API and ensured that all endpoints performed according to expected requirements.